ASSIGNMENT 2 - CO2

TASK 2 REPORT

When Is ArrayList an Appropriate Engineering Choice?

Problem context: Music Playlist Editor

Comparison area	Implemented evidence
Playlist sizes	500, 5,000 and 20,000 songs
Collections	ArrayList, LinkedList and ArrayDeque for the Up Next workload
Operations	Add at end; insert/remove at front and middle; get k-th; full iteration
Repetitions	1 warm-up trial and 5 measured repetitions per benchmark case
Output files	Raw measurements in task2_results.csv; execution log in run_output.txt

Purpose. This report evaluates the data-structure choice for a playlist editor and then re-evaluates it for an Up Next queue. The conclusions are based on the Java benchmark included with this submission.

Execution environment. The supplied benchmark was executed using OpenJDK 21.0.12.1 in the available Linux environment. Absolute timings vary with processor, memory, operating system, JVM and background load. Re-run the program on the submission computer and retain its CSV for machine-specific evidence.

1. Experiment design and method

The assignment asks for a comparison of ArrayList and LinkedList across end addition, front and middle insertion/removal, indexed access and full iteration, using at least three playlist sizes. The changed condition asks whether ArrayDeque is more appropriate when items are added at both ends and removed from the front.

Test data and measurement method

Item	Design used
Playlist sizes	500, 5,000 and 20,000 Song objects. Data is generated deterministically in the Java program; no external input file is needed.
Repeated trials	One warm-up trial followed by five measured repetitions for each size, structure and operation.
End addition	Start with an empty list and append N songs. The timed region includes Song creation and list insertion.
Front/middle changes	Start from a pre-populated list; run 400 insertions or 400 removals at the requested position. Setup is outside the timed region.
Middle position	The middle index is recalculated from the current list size for each operation. List.add(index) and List.remove(index) are used for bo
Get k-th	Access deterministic indices repeatedly; each repetition uses at least 100 accesses. The number of accesses is recorded in the CS
Full iteration	Iterate through the complete list for enough passes to process approximately 100,000 elements per repetition.
Up Next	Start with N items; repeat 500 cycles of addFirst, addLast, removeFirst, removeFirst. Two removals balance the two additions and k
Timing	System.nanoTime() measures the timed operation block. CSV stores each run, elapsed time, operation count, normalized ns/opera

Fairness and interpretation. List creation for positional edits, indexed access and iteration is excluded from the timed region. For end-addition, construction from an empty structure is the operation being measured. The code uses the same generated Song payload for both list types. The Up Next cycle uses each collection's end-specific operations; ArrayDeque is not treated as an indexed List.

Important caution. These are microbenchmarks rather than a formal JMH study. The five-run averages are useful for observing the operation trends requested by the assignment, but very short timings can be affected by JIT compilation, garbage collection and system noise. The raw repeated values are supplied so results can be inspected rather than relying on a single timing.

2.1. Results for playlist size 500

Mean time per operation, averaged over five measured repetitions. Values are normalized nanoseconds per individual operation; for full iteration, one operation means visiting one Song.

Operation	ArrayList (ns/op)	LinkedList (ns/op)	Observed pattern
Add at end	172.09	126.64	Both support end addition efficiently
Insert at front	215.86	170.39	LinkedList avoids shifting the remaining elements
Remove at front	142.02	68.60	LinkedList avoids shifting the remaining elements
Insert in middle	185.44	957.68	LinkedList index traversal is costly
Remove from middle	119.50	687.72	LinkedList index traversal is costly
Access k-th element	66.61	242.63	ArrayList indexed access is lower-cost
Iterate whole list	4.53	4.83	Both are linear; contiguous storage helps ArrayList

At 500 songs, the data set is small. Both structures remain practical. LinkedList performs front edits well, while ArrayList provides faster indexed access and competitive middle-position operations in this index-based test.

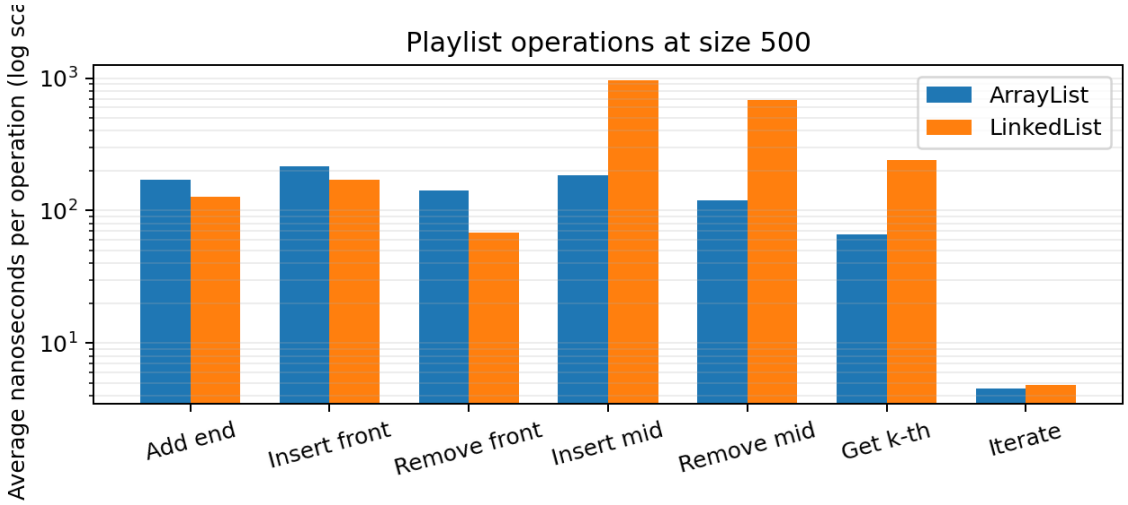


Figure. Average operation costs for this playlist size. The vertical axis is logarithmic so both low-cost and high-cost operations remain visible.

2.2. Results for playlist size 5,000

Mean time per operation, averaged over five measured repetitions. Values are normalized nanoseconds per individual operation; for full iteration, one operation means visiting one Song.

Operation	ArrayList (ns/op)	LinkedList (ns/op)	Observed pattern
Add at end	210.95	47.81	Both support end addition efficiently
Insert at front	292.69	86.23	LinkedList avoids shifting the remaining elements
Remove at front	369.20	76.24	LinkedList avoids shifting the remaining elements
Insert in middle	272.33	5,305.50	LinkedList index traversal is costly
Remove from middle	196.56	5,048.81	LinkedList index traversal is costly
Access k-th element	51.46	2,401.07	ArrayList indexed access is lower-cost
Iterate whole list	2.14	3.96	Both are linear; contiguous storage helps ArrayList

At 5,000 songs, the indexed-access difference is clearer: ArrayList uses direct indexing, while LinkedList must traverse nodes. LinkedList front changes remain inexpensive. Middle insertion/removal through List index methods becomes substantially slower because a node must first be located.

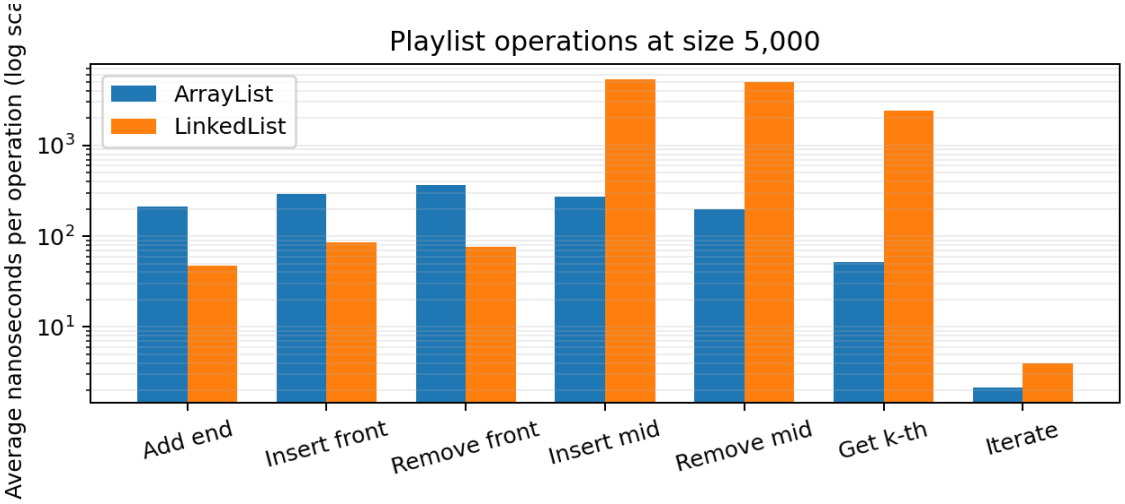


Figure. Average operation costs for this playlist size. The vertical axis is logarithmic so both low-cost and high-cost operations remain visible.

2.3. Results for playlist size 20,000

Mean time per operation, averaged over five measured repetitions. Values are normalized nanoseconds per individual operation; for full iteration, one operation means visiting one Song.

Operation	ArrayList (ns/op)	LinkedList (ns/op)	Observed pattern
Add at end	49.22	33.83	Both support end addition efficiently
Insert at front	1,085.03	93.00	LinkedList avoids shifting the remaining elements
Remove at front	1,044.32	35.97	LinkedList avoids shifting the remaining elements
Insert in middle	636.24	22,723.64	LinkedList index traversal is costly
Remove from middle	739.71	25,191.29	LinkedList index traversal is costly
Access k-th element	57.18	11,543.35	ArrayList indexed access is lower-cost
Iterate whole list	2.13	3.86	Both are linear; contiguous storage helps ArrayList

At 20,000 songs, ArrayList provides markedly lower get(index) and middle-index operation times in this benchmark. LinkedList retains the advantage for front insertion/removal. For a playlist UI that frequently jumps to a song by index, ArrayList is the stronger general-purpose choice.

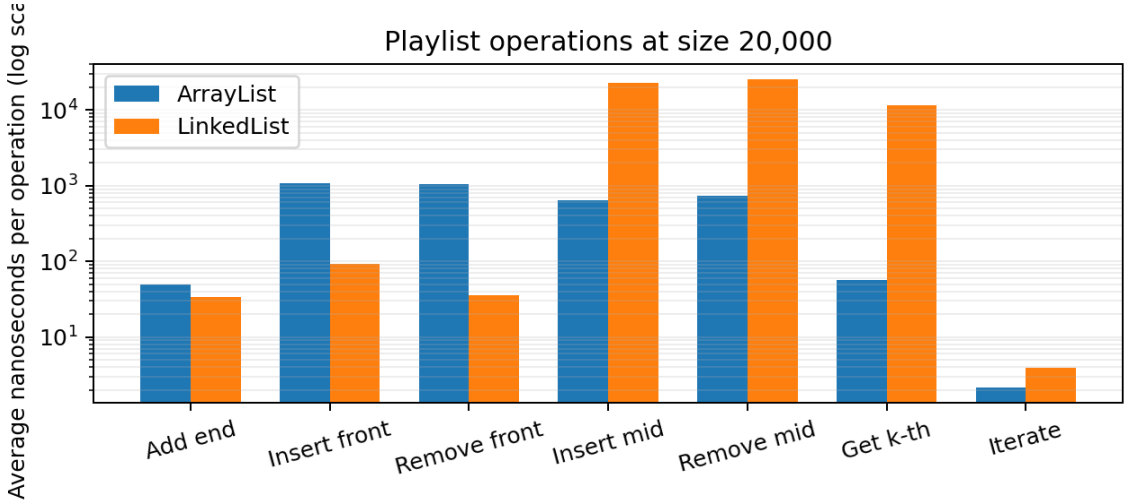


Figure. Average operation costs for this playlist size. The vertical axis is logarithmic so both low-cost and high-cost operations remain visible.

3. Changed condition: the Up Next queue

The app now frequently adds songs at both ends and removes from the front. The benchmark starts each queue at the given playlist size and measures 500 balanced cycles. Each cycle performs addFirst, addLast, removeFirst and removeFirst. That is four queue operations per cycle.

Initial size	ArrayList (ms / 500 cycles)	LinkedList (ms / 500 cycles)	ArrayDeque (ms / 500 cycles)
500	0.2230	0.1744	0.1823
5,000	0.4035	0.0973	0.0963
20,000	1.5368	0.0847	0.0891

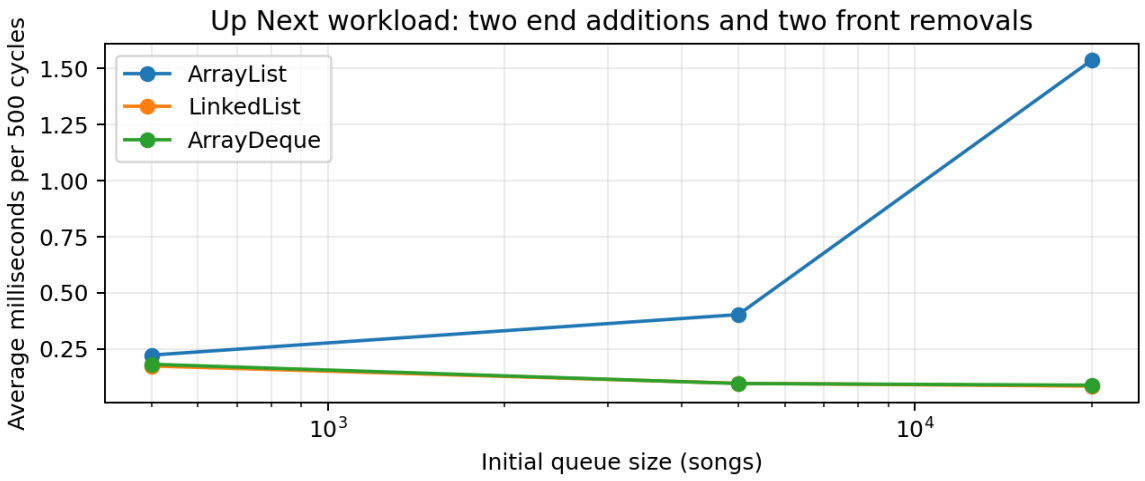


Figure 2. The Up Next workload shows ArrayList cost increasing with queue size because front insertions/removals shift elements. LinkedList and ArrayDeque stay comparatively stable for end operations.

Recommendation. Use ArrayDeque for the Up Next queue. It is designed for efficient operations at both ends and avoids per-element linked-node overhead. LinkedList also supports the required end operations, but ArrayDeque is the more direct queue/deque choice. Keep ArrayList for the main playlist when random indexed access is important.

Why not one structure for both? The main playlist and the Up Next queue have different access patterns. A single collection does not need to serve both workloads. The playlist benefits from indexed access; the queue benefits from efficient deque-end operations.

4. Complexity and space comparison

The table below gives the expected asymptotic costs for the operations in the assignment. For LinkedList middle edits, the position must first be located; linking/unlinking the node is O(1) after the location is known. For ArrayDeque, there is no indexed List API or direct middle insertion.

Operation	ArrayList	LinkedList	ArrayDeque
Add at end	Amortized O(1)	O(1) at tail	Amortized O(1) at tail
Insert at front	O(n) shifts	O(1) at head	Amortized O(1) at head
Remove from front	O(n) shifts	O(1) at head	O(1) at head
Insert in middle	O(n) shifts	O(n) locate + O(1) link	No indexed API; O(n) traversal/rebuild
Remove from middle	O(n) shifts	O(n) locate + O(1) unlink	No indexed API; O(n) traversal/rebuild
Get k-th	O(1)	O(n)	O(n) by traversal; no get(index)
Iterate all	O(n)	O(n)	O(n)
Space per element	One reference slot; spare capacity may exist	One node with item + previous/next links	One reference slot in circular array; spare capacity may exist

Plain array with manual shifting

A playlist can also be stored in a manually managed Song[] array plus a size variable. Insertion at index shifts elements one position to the right; removal shifts later elements one position to the left. The array must have free capacity or be enlarged when full.

```
insertAt(index, song):
  ensureCapacityIfFull()
  for i = size down to index + 1: data[i] = data[i - 1]
  data[index] = song; size = size + 1

removeAt(index):
  removed = data[index]
  for i = index to size - 2: data[i] = data[i + 1]
  size = size - 1; data[size] = null
  return removed
```

Complexity: get(index) is O(1); iteration is O(n); insert/remove at front or middle is O(n) because elements must be shifted. Append is O(1) while capacity remains, or amortized O(1) if the array grows geometrically. Its space cost is O(n) references plus any unused capacity. This behaves similarly to ArrayList, but the developer must implement resizing, shifting and bounds checks manually.

5. Engineering decision and conclusion

When ArrayList is reasonable

ArrayList is a sound engineering choice for the main playlist when users frequently jump to the k-th song, browse/iterate through tracks, and append new songs, while front or middle edits are relatively occasional. Its contiguous reference storage and O(1) indexed access are valuable for this usage pattern.

When to stop recommending ArrayList

Do not recommend ArrayList as the primary structure for a workload dominated by repeated insertions/removals at the front of a large list. Each front edit shifts the remaining references. The 20,000-song measurements show this cost increasing compared with LinkedList. For frequent middle edits, a LinkedList can be useful only when the application already has an iterator positioned at the edit point; searching to a middle index remains O(n), so simply replacing ArrayList with LinkedList does not make indexed middle edits constant-time.

Final collection choice

Application need	Recommended structure	Reason
Main playlist with direct song-number access	ArrayList	O(1) indexed access and efficient sequential traversal; occasional shifts are acceptable.
Up Next queue with frequent operations at both ends	ArrayDeque	Efficient add/remove at both ends with array-backed storage; no per-song linked node.
Frequent front edits where linked nodes are useful	LinkedList	O(1) operations at head/tail; avoid index-based searches when possible.

Limitations and validation note

The timings are specific to the execution environment and should not be treated as universal constants. Five repeated measurements and raw CSV values are included, but the benchmark is not a substitute for a full JMH study. For the final submission, run PlaylistBenchmark.java on the computer used for coursework, retain the newly generated task2_results.csv and take a screenshot of the console summary. Use those local measurements if they differ materially from this report.

Conclusion. ArrayList is appropriate for the main Music Playlist Editor under an access-heavy or append-heavy pattern, but it becomes less attractive when front edits dominate as the playlist grows. The changed Up Next condition favours ArrayDeque because the required actions are deque operations at both ends. The best engineering design is therefore to select a collection based on the operations each component performs rather than force every component to use the same structure.

Files supplied with this report: PlaylistBenchmark.java, task2_results.csv (raw repeated measurements), run_output.txt (console execution log) and run instructions in README.md.